Funktionen und Rekursion

DEFINIEREN UND AUFRUFEN

```
def addiere(a: int, b: int) -> int:
    ergebnis = a + b
    return ergebnis

print(addiere(5, 7))  # 12
```

Teil	Bedeutung
`def`	Definition beginnt
`a`, `b`	Parameter (Platzhalter)
`5`, `7`	Argumente (Werte beim Aufruf)
`-> int`	Rückgabetyp, prüft nur ty
`return`	beendet, liefert Ergebnis

Merksatz: `def` legt nur an, erst `name(...)` führt aus.

```
def main() -> None:
    print("Start")

if __name__ == "__main__":
    main()  # nur bei direktem Start
```

PRINT ODER RETURN?

	`print(x)`	`return x`
zeigt etwas an?	ja	nein
weiterrechnen möglich?	nein (`None`)	ja
beendet Funktion?	nein	ja, sofort

```
def f() -> None:
    print(119.0)

a = f()     # zeigt 119.0
print(a)    # None
```

- Ohne `return` (auch in einem Zweig): `None`.
- Mehrere Werte = Tupel: `return min(z), max(z)`, dann `k, g = ...`.
- `break` verlässt die Schleife, `return` die ganze Funktion.

GÜLTIGKEIT UND PARAMETER

- Variablen in der Funktion sind **lokal** (eigener Rahmen auf dem Aufrufstapel).
- `if` / `for` bilden keinen eigenen Bereich, Funktionen schon. Suche: LEGB.
- Globale Variable in der Funktion zuweisen: `UnboundLocalError`.
- Parameter zeigt auf **dasselbe Objekt** wie das Argument.

```
def aendere(liste: list[int]) -> None:
    liste[0] = 99      # außen sichtbar
    liste = [7, 7, 7]  # nur lokal

z = [1, 2, 3]
aendere(z)
print(z)  # [99, 2, 3]
```

STANDARDWERTE STATT ÜBERLADUNG

Keine Überladung: Zweites `def` mit gleichem Namen ersetzt das erste.

```
def brutto(netto, satz=0.19):
    return round(netto * (1 + satz), 2)

print(brutto(100))            # 119.0
print(brutto(100, 0.07))      # 107.0
print(brutto(satz=0.07, netto=100))  # 107.0
```

```
def summe(*zahlen: int) -> int:
    ergebnis = 0
    for zahl in zahlen:
        ergebnis += zahl
    return ergebnis

print(summe(1, 2, 3, 4))  # 10
```

Standardwert-Parameter stehen hinten. Liste als Standardwert: stattdessen `None` und in der Funktion `[]`.

IMPERATIV UND APPLIKATIV

	imperativ	applikativ
Frage	Wie?	Was ist das Ergebnis?
Zustand	ändert sich	bleibt
Wiederholung	`for`, `while`	Rekursion

REKURSION

Pflicht: **Abbruchbedingung** und **Aufruf mit kleinerem Problem**.

```
def fakultaet(n: int) -> int:
    if n == 0:        # Basisfall
        return 1
    return n * fakultaet(n - 1)

print(fakultaet(3))  # 6
```

Aufruf	wartet auf	liefert
fakultaet(3)	fakultaet(2)	$3 \cdot 2 = 6$
fakultaet(2)	fakultaet(1)	$2 \cdot 1 = 2$
fakultaet(1)	fakultaet(0)	$1 \cdot 1 = 1$
fakultaet(0)	–	1

Tabelle von unten nach oben lesen. Ohne Abbruch: `RecursionError` (ca. 1000 Rahmen).

TESTEN

```
assert fakultaet(5) == 120  # nichts
assert fakultaet(3) == 7    # AssertionError
```

pytest: `uv add --dev pytest`, Datei `test_….py`, Funktionen `test_…`, `uv run pytest`. Randfälle mittesten (leere Liste, Exponent 0).

TYPISCHE FEHLER

- `print` statt `return`: Ergebnis ist `None`.
- `main` ohne Klammern: nichts passiert.
- Fehlendes `return` in einem Zweig.
- Zwei `def addiere`: `TypeError` beim Aufruf, kein `SyntaxError`.
- Liste als Standardwert: wird zwischen Aufrufen geteilt.
- Basisfall `return 0` bei der Fakultät: alles wird 0.
- `fakultaet(-1)`: läuft an `n == 0` vorbei, `RecursionError`.
- Nur ein Test: kann zufällig bestehen.